SOFTWARE SEGURO DESDE EL DISEÑO

LA SEGURIDAD SE DECIDE ANTES DE ESCRIBIR CÓDIGO.

Requisitos → **Diseño** → Código → Operación

Las decisiones de diseño condicionan lo que podremos proteger después.

¿QUÉ ES UNA VULNERABILIDAD?

Una debilidad en un sistema, sus procedimientos, controles o implementación que una amenaza puede explotar o activar.

Puede estar en cualquier artefacto del SDLC:



Requisitos e historias



Diseño y arquitectura



Código y dependencias



Scripts, pipeline y build



Configuración e infraestructura



Software en operación

Una vulnerabilidad no está limitada al código fuente.

NUESTRO CASO: UNA APP DE BANCA DIGITAL

Un mismo sistema nos acompañará toda la clase: autenticarse, consultar y transferir.



Autenticarse

El cliente ingresa con su cuenta y verifica su identidad.



Consultar

Mira el saldo y los movimientos de su cuenta.



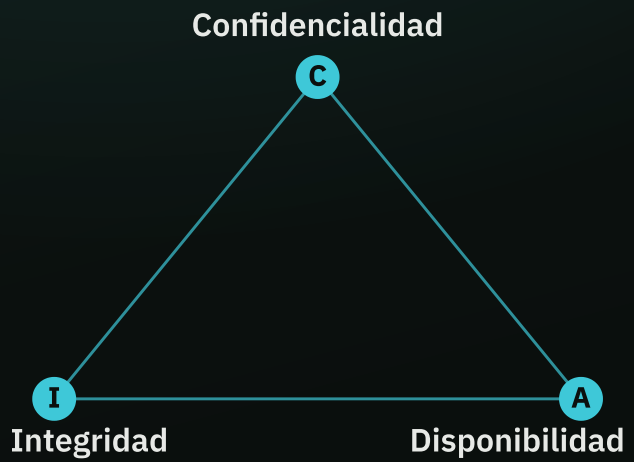
Transferir

Inicia una transferencia a otra cuenta.

Detrás de la app hay servicios, una base de datos y un proveedor externo de notificaciones.

¿QUÉ QUEREMOS PROTEGER?

Activo: algo que tiene valor para una persona u organización. **En la app de banca:** los datos de clientes, el registro de movimientos y la disponibilidad del servicio.



Confidencialidad

El saldo y los movimientos solo son visibles para el titular de la cuenta.

Integridad

El importe y el destinatario de una transferencia permanecen correctos.

Disponibilidad

La app responde cuando el cliente necesita hacer una transferencia.

Primero identificamos el valor y las propiedades que lo sostienen; después, qué podría ponerlo en riesgo.

AMENAZA Y ATACANTE

Amenaza: qué podría ocurrir

Circunstancia o evento con potencial de dañar activos, personas u operaciones.

Atacante: quién actúa con intención

Persona o grupo que intenta comprometer la seguridad del sistema, desde dentro o fuera de la organización.

Ejemplos en la app de banca:

Amenaza	Quién o qué la origina	¿Hay atacante?
Consulta no autorizada de movimientos	Persona que busca datos ajenos	Sí
Exposición accidental de datos	Administrador que configura mal un permiso	No
Interrupción de las transferencias	Falla de la base de datos	No

No toda amenaza requiere un atacante.

CONSECUENCIAS REALES DE CIBERATAQUES

Patrones en brechas observadas globalmente (Verizon DBIR 2026): **31%** se inició explotando vulnerabilidades · **48%** incluyó ransomware · **48%** involucró a terceros.

SECTOR PÚBLICO · INGLATERRA · 2017

WANNACRY Y EL NHS

Más de 19.000

citas estimadas canceladas; en cinco zonas, hospitales derivaron pacientes a otros servicios de urgencias.

SECTOR PRIVADO · ESTADOS UNIDOS · 2021

COLONIAL PIPELINE

Combustible interrumpido

Un ataque a sistemas corporativos llevó a detener temporalmente el oleoducto por precaución.

Las técnicas se repiten; el impacto alcanza a personas y servicios que dependen de los sistemas afectados.

DE LA AMENAZA AL RIESGO

Una amenaza es un escenario con potencial de daño. El riesgo valora si ese escenario puede concretarse en **este** sistema y cuánto afectaría.

Amenaza

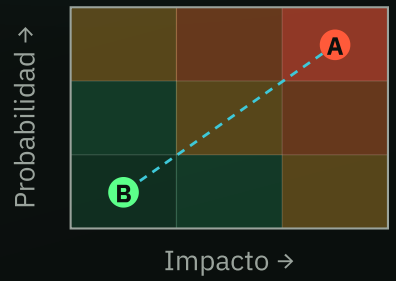
El escenario: qué podría pasar y quién o qué podría provocarlo.

Vulnerabilidad y exposición

Deciden si el escenario puede concretarse: una debilidad y un punto de contacto que la alcance.

Controles

Reducen la probabilidad de que ocurra, el impacto si ocurre, o ambos.



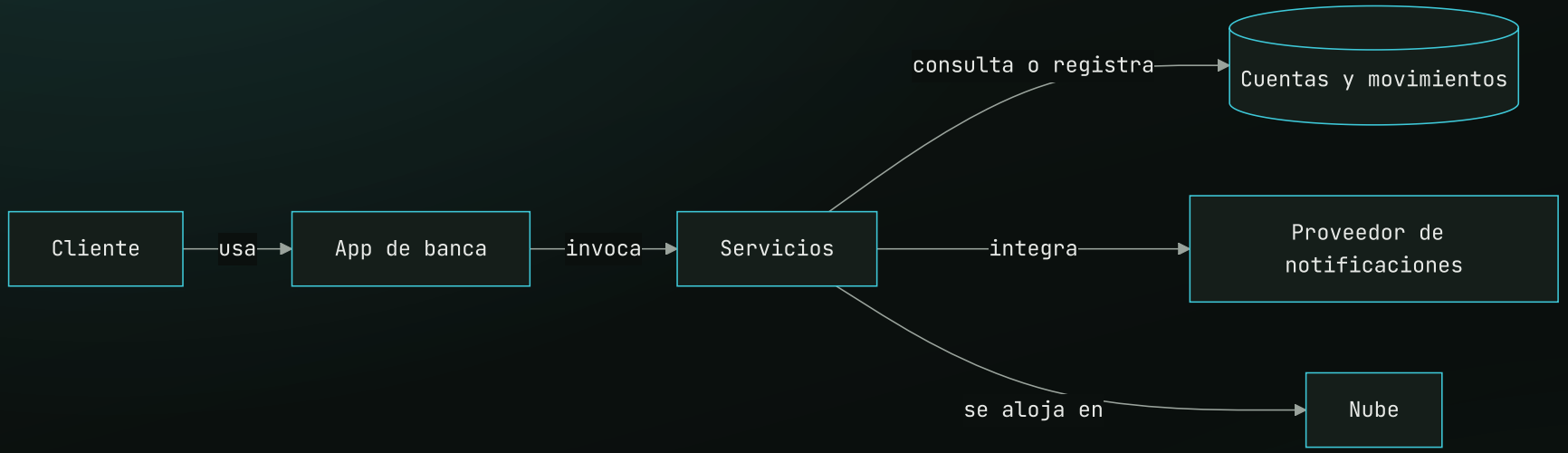
Riesgo ≈ probabilidad × impacto

¿Qué tan probable es aquí, y cuánto afectaría? Una aproximación para ordenar la conversación, no una fórmula.

A sin controles · B con controles: el mismo escenario se desplaza.

La misma amenaza puede dar riesgos distintos: cambian las vulnerabilidades, la exposición y los controles.

EL SOFTWARE MODERNO ESTÁ CONECTADO



Cada vínculo requiere definir qué se intercambia, quién participa y qué permisos necesita. Proteger solo la entrada de la red no alcanza:

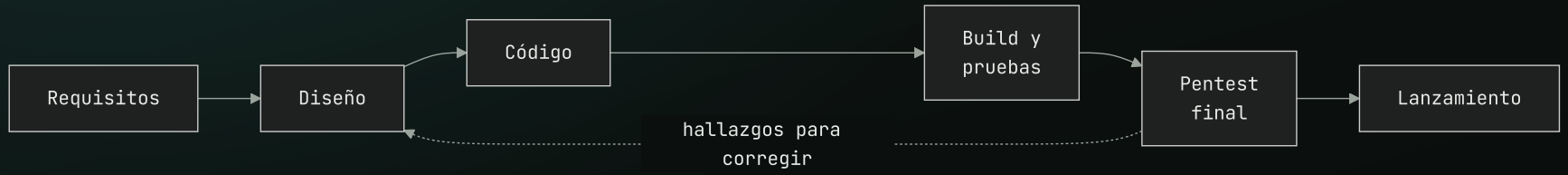
Identidad
Una cuenta legítima puede usarse mal o ser robada.

Servicios
Un componente conectado puede quedar comprometido.

Comunicaciones
Cada llamada requiere controles propios.

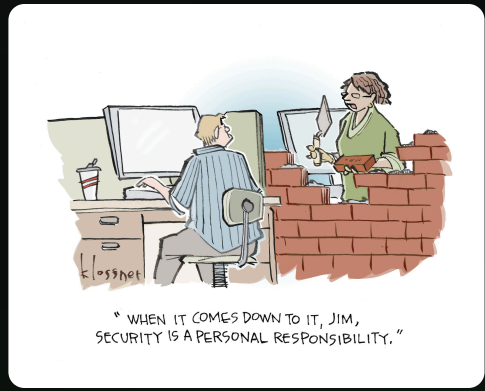
EL LÍMITE DE LA SEGURIDAD REACTIVA

El modelo de **fortaleza y foso** concentra la defensa en la red y deja la seguridad de la aplicación para una revisión antes del lanzamiento.



Una revisión tardía puede sumar retrabajo y tensionar la fecha de liberación.

La seguridad no depende de una herramienta aislada: es una **propiedad del sistema**, sostenida por decisiones coordinadas en **diseño, implementación, configuración y operación**.

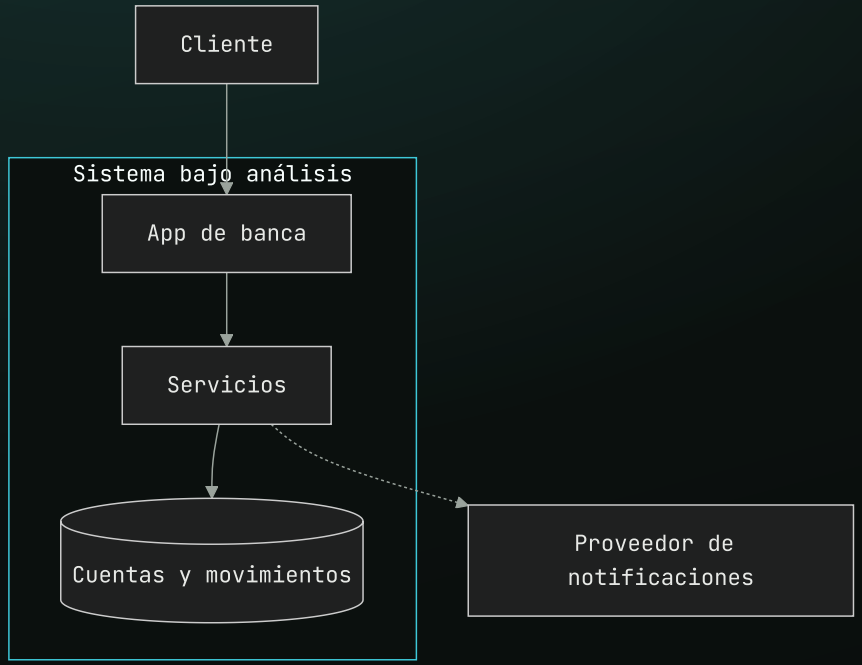


EL ALCANCE DEL SISTEMA DEFINE EL ANÁLISIS



Fuera de nuestro control no significa fuera del análisis.

DIBUJEMOS EL SISTEMA

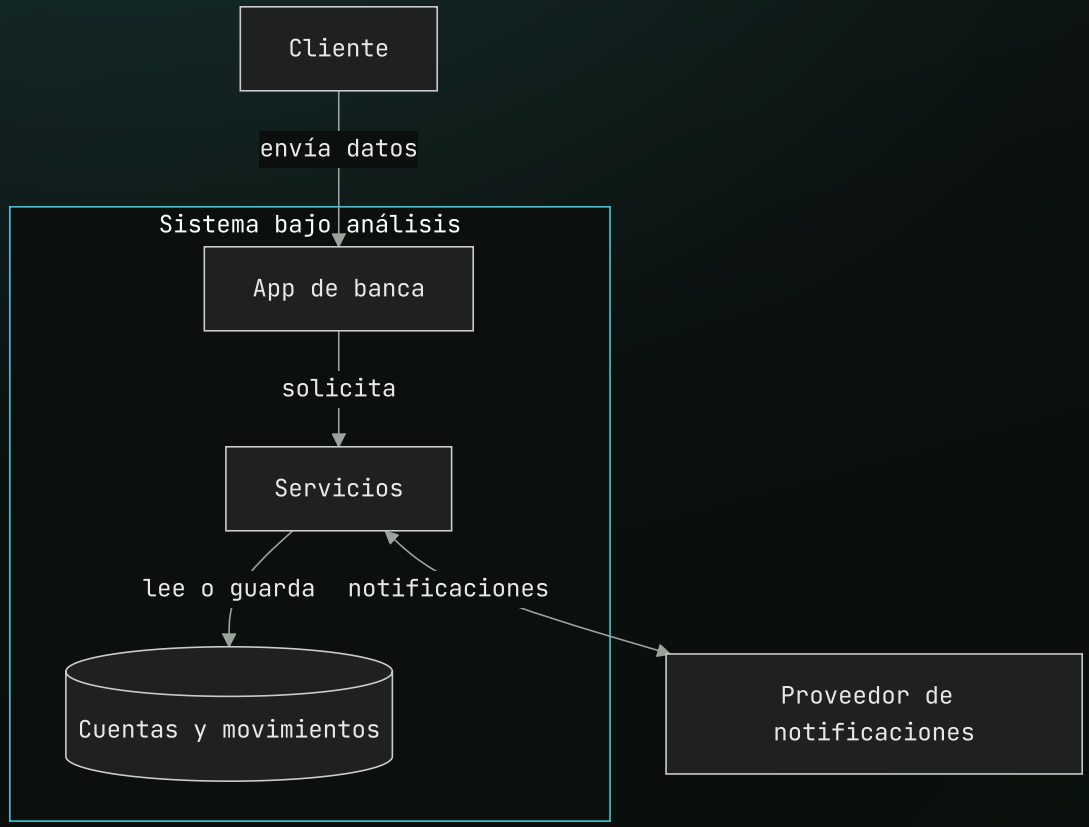


El recuadro marca el alcance; la línea punteada, una dependencia externa.

COMPONENTES, EN NUESTRO CASO

Cliente	persona con una cuenta
App de banca	app móvil o web
Servicios	autenticación, extractos, pagos
Cuentas y movimientos	saldos e historial
Proveedor de notificaciones	confirmaciones por SMS o correo

LA INFORMACIÓN SE MUEVE



QUÉ IDENTIFICAR

Origen
Persona o sistema que genera la información.

Destino
Servicio, almacén o sistema que la recibe.

Acceso
Quién puede leerla o modificarla.

¿EN QUIÉN CONFIAMOS?

Confianza: expectativa de que una persona o componente se comporte como esperamos, para una tarea y en un contexto concretos.

No basta con asumirla: necesitamos evidencia y límites. Tres comprobaciones complementarias:

Autenticación

Verificar la identidad declarada.

El cliente inicia sesión con sus credenciales y un segundo factor.

Autorización

Comprobar el permiso para una acción sobre un recurso.

Puede consultar su cuenta, no la de otro cliente.

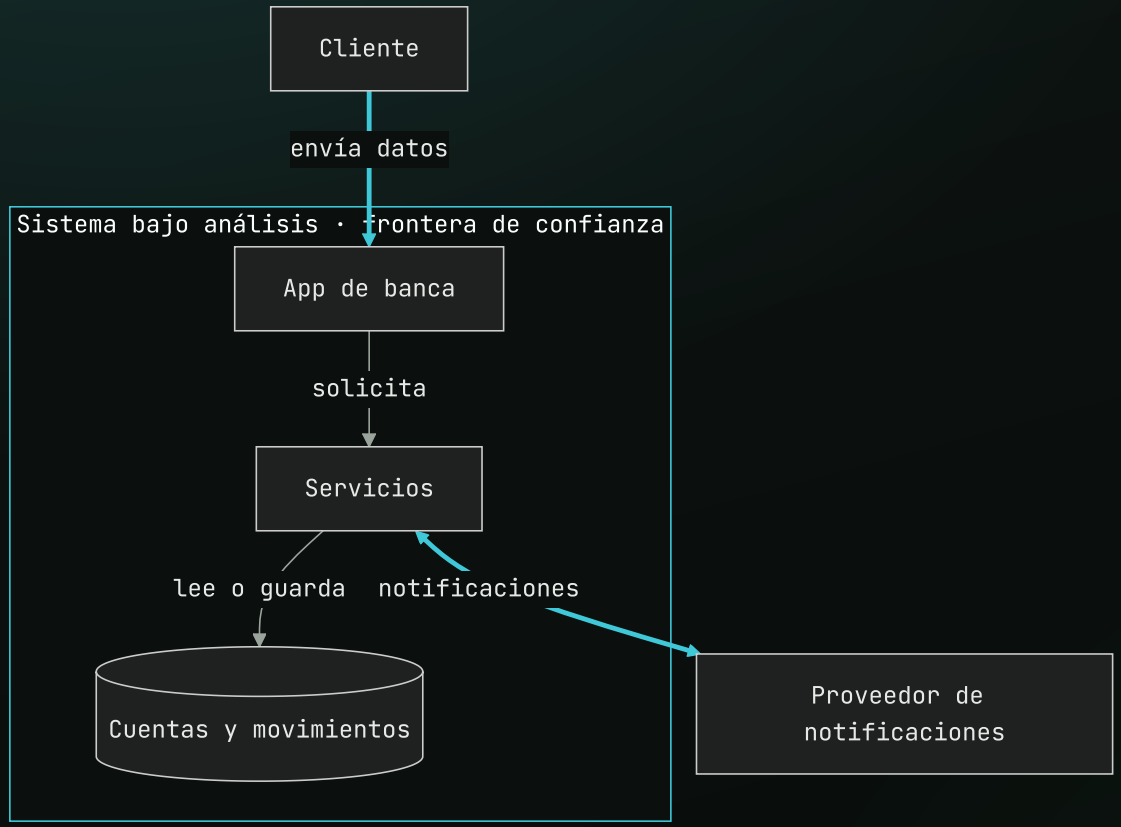
Validación de datos

Comprobar formato, valores y reglas de la operación.

El importe de una transferencia debe ser válido, aunque el cliente ya ingresó.

Autenticarse no da permiso para todo ni vuelve confiable cualquier dato enviado.

FRONTERAS DE CONFIANZA



Una frontera de confianza marca el paso entre contextos con distintos niveles de confianza. Su nombre técnico es **trust boundary**.

AL CRUZARLA, VALIDAMOS

Identidad	quién envía: autenticación
Permisos	qué puede hacer: autorización
Datos	que sean válidos: validación de entrada

SUPERFICIE DE ATAQUE

Son los lugares donde alguien puede interactuar con el sistema o influir en él.



Entradas

formulario de transferencia



Cuentas

credenciales de clientes



APIs

endpoints de saldo y pagos



Archivos

extractos descargables



Interfaces

pantallas de la app



Servicios externos

proveedor de notificaciones

Reducir entradas innecesarias reduce oportunidades de abuso.

¿DE DÓNDE VIENEN LAS BRECHAS?

CAUSAS FRECUENTES

Phishing y amenazas internas
Intencionadas o accidentales.

Software de terceros
Vulnerable o sin parches.

Configuración incorrecta
Especialmente en la nube.

LA SUPERFICIE SE EXPANDE

Nuevos servicios digitales
En banca, por ejemplo: home banking, onboarding digital, pagos instantáneos.

Arquitecturas modernas
Microservicios, contenedores y nube suman dependencias.

PRINCIPIOS DE DISEÑO SEGURO

No hace falta memorizar una lista extensa. Empecemos con preguntas útiles:



¿Quién necesita este acceso?

Confiar lo mínimo necesario.



¿Qué pasa si una defensa falla?

Varias defensas; fallar de forma segura.



¿Cómo limitamos el daño?

Separar y contener.

CONFIAR LO MÍNIMO NECESARIO

¿Qué permisos necesita cada identidad para su tarea, y por cuánto tiempo?



Alcance

El servicio de extractos consulta saldos y movimientos, pero no puede iniciar transferencias.



Duración

El acceso de mantenimiento a la base de datos se habilita por ventana y se revoca al terminar.

Limitar permisos reduce lo que una cuenta comprometida puede hacer.

NO DEPENDER DE UNA ÚNICA DEFENSA

¿Qué pasa si una barrera falla?

La defensa en profundidad combina controles complementarios.



Prevenir

Validar entradas y limitar accesos.



Detectar

Registrar actividad y alertar ante anomalías.



Contener

Aislar componentes y acotar permisos.

Si un control se supera, otros todavía pueden detectar el incidente o limitar su impacto.

DISEÑAR TAMBIÉN PARA CUANDO ALGO FALLE

Si un control o una dependencia falla, ¿cómo debería responder el sistema?



Proteger

Sin poder verificar autorización, no ejecutar transferencias.



Degradar

Sin notificaciones, seguir consultando y transfiriendo.



Recuperar

Registrar el fallo y restablecer el servicio de forma controlada.

Fallar seguro no es apagar todo: es limitar las acciones sensibles y conservar, cuando sea posible, las funciones independientes del componente fallido.

LIMITAR EL IMPACTO DE UN COMPROMISO

Si se compromete una cuenta, ¿hasta dónde puede llegar?



Alcance amplio

Una cuenta con permisos amplios puede leer movimientos, iniciar transferencias y administrar usuarios.



Acceso acotado

El servicio de extractos solo lee; el de pagos escribe transferencias; administración usa permisos separados.

Separar permisos y componentes reduce el radio de impacto (*blast radius*).

¿CÓMO SE PODRÍA ABUSAR DEL SISTEMA?

Uso esperado

Consultar el saldo y los movimientos de la propia cuenta.

→

Intento de abuso

Modificar el identificador de la cuenta para intentar consultar movimientos ajenos.

Mirar más allá del uso esperado revela requisitos que el camino feliz no muestra.

THREAT MODELING

¿Qué protegemos, qué podría salir mal y cómo respondemos?



Entender

Actores, datos, componentes y flujos.



Analizar

Posibles abusos y fallos.

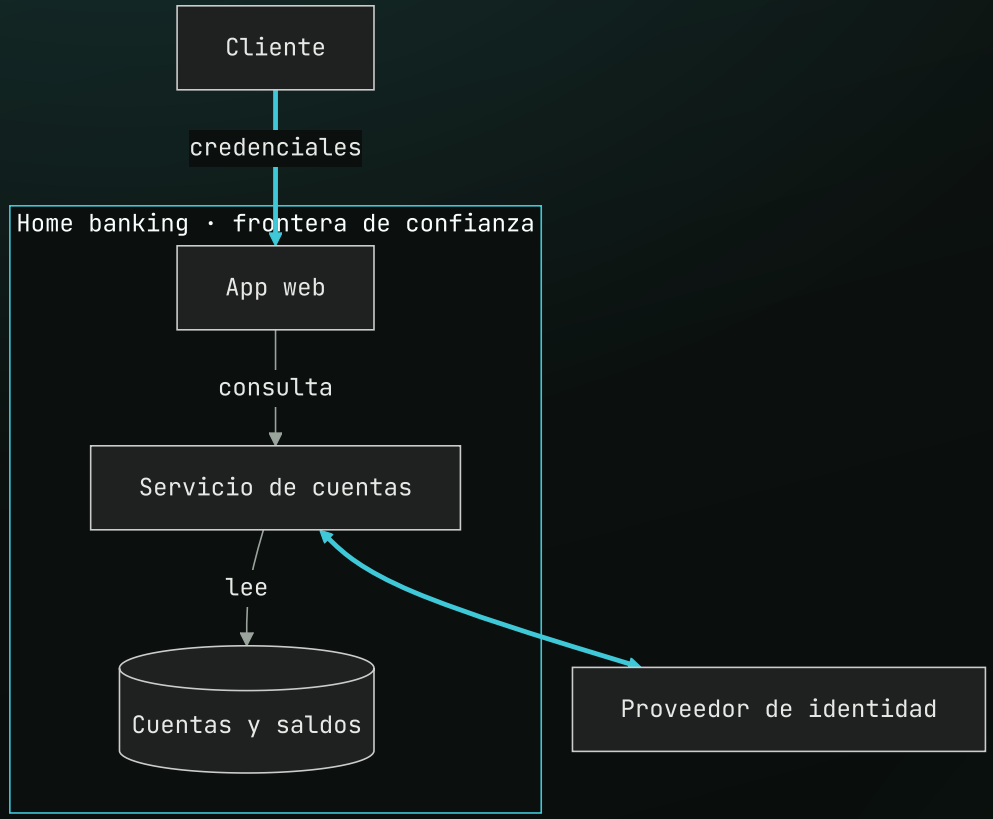


Responder

Requisitos, controles y pruebas.

Threat Modeling ordena esta conversación; no requiere una herramienta específica.

¿QUÉ PODRÍA SALIR MAL?



PREGUNTAR COMO ATACANTE

Actor	¿quién intenta algo?
Acción	¿qué intenta hacer?
Objetivo	¿qué gana si logra?

En juego
Credenciales, saldos y movimientos, disponibilidad del servicio.

El sistema es el mismo: cambia quién lo mira y con qué intención.

STRIDE: SEIS TIPOS DE AMENAZA

Seis lentes para explorar posibles abusos en actores, flujos y componentes.

S **Suplantación**

Usar una identidad ajena.

T **Manipulación**

Alterar el importe o el destinatario de una transferencia.

R **Repudio**

Negar una acción sin evidencia trazable.

I **Exposición de información**

Leer movimientos de otra cuenta.

D **Denegación de servicio**

Impedir consultas legítimas.

E **Elevación de privilegios**

Obtener permisos mayores a los asignados.

¿QUÉ LENTE NOS FALTÓ?

Cada letra es una pregunta para revisar la lista de amenazas, no un orden de gravedad.

S **Suplantación**

¿Alguien puede actuar con una identidad ajena?

T **Manipulación**

¿Alguien puede alterar una solicitud o un registro?

R **Repudio**

¿Alguien puede negar una acción sin evidencia?

I **Exposición de información**

¿Alguien puede leer datos de otra cuenta?

D **Denegación de servicio**

¿Alguien puede impedir el servicio legítimo?

E **Elevación de privilegios**

¿Alguien puede obtener permisos de más?

La letra que no aparece en la lista señala una amenaza que nadie vio.

ENCONTRAR UNA AMENAZA NO ALCANZA

¿Qué hacemos con lo que encontramos?

La respuesta depende del escenario y de lo que está en juego.



Cambiar el diseño

Eliminar o reducir el escenario de abuso.



Agregar un control

Prevenir, detectar o limitar el impacto.



Aceptar conscientemente

Documentar por qué el riesgo es tolerable.

La categoría de amenaza, por sí sola, no determina la prioridad.

ELIJAMOS UNA AMENAZA

Una sola amenaza de la lista, seguida hasta su comprobación.

Amenaza

¿Quién hace qué y para qué?

Requisito

¿Qué debe garantizar siempre el sistema?

Control

¿Dónde y quién lo verifica?

Prueba

¿Qué resultado demuestra que funciona?

Cada paso se responde con una sola oración.

DEL USO ESPERADO AL ABUSO

¿Quién hace qué y con qué objetivo?

Uso esperado

Como **cliente**, quiero **consultar el saldo y los movimientos de mi cuenta** para **administrar mi dinero**.

Historia de abuso

Evil User Story

Como **atacante**, quiero **consultar el saldo y los movimientos de otra cuenta** para **obtener información privada**.

Escribir el abuso revela requisitos que el camino esperado no muestra.

DEL ABUSO AL REQUISITO

Amenaza

Una persona intenta consultar el saldo y los movimientos de otra cuenta.

→

Requisito de seguridad

En cada consulta, el sistema verifica que la identidad autenticada esté autorizada para esa cuenta.

El requisito define qué debe garantizarse, no cómo implementarlo.

INTEGRACIÓN TEMPRANA

La seguridad se considera junto con las necesidades funcionales, no al final.



Historia de usuario · Historia de abuso · Historia de seguridad: las tres se discuten en la misma planificación.

Requisitos, arquitectura y controles se deciden antes de implementar.

DE LA AMENAZA A LA PRUEBA

Amenaza	Requisito	Control	Prueba
Consultar el saldo y movimientos de otra cuenta.	Verificar autorización en cada consulta.	El servidor valida el permiso sobre la cuenta solicitada.	Otra identidad recibe rechazo y no obtiene los datos.

La trazabilidad conecta el escenario con una garantía y una comprobación.

ESAS PROPIEDADES NO SE CONSERVAN SOLAS

El diseño define las reglas; el resto del ciclo debe sostenerlas.



A este enfoque se lo llama SSDLC: la seguridad acompaña todo el ciclo de vida.

PERO EL SOFTWARE CAMBIA

Cambian el código, las dependencias, la configuración y la infraestructura.

- Código
- Dependencias
- Configuración
- Infraestructura

Cada cambio puede alterar lo que el sistema permite hacer.

UNA PREGUNTA PARA LA PRÓXIMA CLASE

¿Cómo verificamos continuamente que esas propiedades se mantienen mientras el software evoluciona?

En la próxima clase

Shift Left or get hacked: verificar la seguridad a medida que cambian el código, las dependencias, la infraestructura y la configuración.

EXAMEN DE LA CLASE

Escaneá el QR o abrí el enlace que corresponda a tu comisión.

QR EXAMEN TALLER



Abrir Examen Taller

QR EXAMEN CUC



Abrir Examen CUC

Disponibilidad para ambos exámenes

SE HABILITA

Viernes 9 de octubre

08:00 GMT-3

SE DESHABILITA (TERMINA)

Sábado 10 de octubre

08:00 GMT-3